COLET FAN SUPORTA

ROADMAP · REFUND / UPGRADE / DOWNGRADE TRACK

# Refund, Upgrade & Downgrade — Plan

Everything already shipped, what's next, and how each piece composes with the ones we already have. Ordered by leverage: we finish the payment-side loop before adding notifications and adjacent workflows.

| OWNER | AS OF |
| --- | --- |
| Colet Fan Suporta engineering | September 2026 |

| PAYMENTS | EMAILS |
| --- | --- |
| PayMongo (QR Ph active; Refunds API status: **to verify**) | Resend (delivered/opened/bounced tracking live) |

## WHERE WE ARE TODAY

### 0 · Shipped foundation DONE

- **Refunds table & admin queue** — `/admin/refunds`, per-row PENDING → COMPLETED workflow, audit-logged.
- **Tier change API** — `POST /api/events/tier-change` handles three cases:
  - Same price → instant swap
  - Upgrade → PayMongo checkout for delta only
  - Downgrade → pending refund row with `[tier_change_target:UUID]` marker
- **Member self-service UI** — `TierChangeBlock` on `/members/tickets/[id]`. Guards for owner, active ticket, non-bundle, ≥12h before event, tier slot availability.
- **Webhook handles `tier_upgrade` type** — on payment success, swaps `event_tickets.tier_id` using metadata.
- **Refund completion → tier swap** — when admin marks a downgrade refund complete, the marker in `note` triggers a tier update instead of a cancel.
- **Waitlist auto-promote helper** — `promoteNextWaitlist(eventId, n)` ready to reuse.

## WHAT SHIPS NEXT, IN ORDER

## 1 · PayMongo auto-refund NEXT

Closes the whole downgrade loop end-to-end.

**Goal:** One click on any pending refund row → we call PayMongo's Refunds API → webhook confirms → we mark completed → downstream side-effects fire (tier swap / ticket cancel / etc.). No more bouncing to PayMongo's dashboard.

**Blocker to verify first:** is your PayMongo account's `/v1/refunds` endpoint enabled? QR Ph refunds especially can be gated. Email PayMongo support before build:

```
Subject: Enable Refunds API for our account
Merchant ID: [your PayMongo merchant id]
Please enable the /v1/refunds endpoint for our account,
including QR Ph payment sources. We need to process
refunds programmatically from our admin dashboard.
```

**Files to build (~half a day once enabled):**

- `POST /api/admin/refunds/paymongo/process` — reads refund row, calls PayMongo API, stores `paymongo_refund_id`, moves status to `processing`
- Webhook handler for `refund.succeeded` / `refund.failed` → marks refund `completed` or `failed`
- **PROCESS VIA PAYMONGO** button on `/admin/refunds` row (pending only, hard confirm)

**Behavior:**

- Original charge older than 180 days → button disabled, tooltip explains
- Amount validation — can only refund up to original charge
- Partial refunds supported (already how downgrades are amounts)
- PayMongo failure → row moves to `failed` with reason, admin gets notified

## 2 · Ticket cancel flow NEXT

Reuses everything from step 1. Answers the "pag ni-cancel yung ticket" question.

**Goal:** Wrap ticket cancellation in one admin action that does everything correctly.

**Currently:** 5 steps across 3 places. Cancel in DB → refund via PayMongo dashboard → log at `/admin/refunds` → email member manually → hope the seat gets used again.

**After:** One CANCEL button on the ticket. Backend does all of this:

1. Sets ticket status = `cancelled`, payment_status = `refunded`
2. Auto-refunds via step 1's endpoint (if PayMongo enabled)
3. Auto-promotes the next waitlist entry using `promoteNextWaitlist(eventId, 1)`
4. Sends the member a branded cancellation email (via existing shell pattern)
5. Frees the capacity count so registration re-opens if it was full

**Files to build:**

- `POST /api/admin/events/tickets/cancel` orchestrator
- `lib/emails/ticket-cancelled.ts` shell (same branding as event-ticket)
- CANCEL button on `/admin/events/[id]/tickets` per row

## 3 · Member notifications on refund lifecycle NEXT

Slots in naturally alongside 1 & 2. Zero new plumbing — reuses the Resend + in-app notifications infra we already have.

**Missing today:** Member submits a downgrade request, then hears nothing until they check back.

**Events to wire:**

- Refund created (queued) → in-app notif "Your refund request is being processed" + optional email
- Refund succeeded → in-app + email "Your ₱X refund has been sent to your original payment method. Expect it in 3–5 business days."
- Refund failed → in-app + email "We couldn't process your refund automatically. Our team will follow up within 24h."
- Tier upgrade paid → in-app "Your ticket has been upgraded to [Tier]. New QR is ready."

**Uses:** Existing `notifications` table, existing `lib/emails/*` shells, PayMongo refund webhook from step 1.

## 4 · Refund performance / reconciliation panel POLISH

Small quality-of-life. Only worth it once volume warrants.

**On `/admin/refunds`:**

- Header tiles: TOTAL REFUNDED THIS MONTH · PENDING · FAILED NEEDS FOLLOWUP
- Amber "stale" flag on any refund row that's been `pending` for > 7 days
- Reason breakdown pie (tier downgrade / cancellation / order / donation)
- CSV export for accounting reconciliation

---

### FLOW: CURRENT VS. AFTER STEP 1 + 2

**Now — tier downgrade (5 manual actions):**

```
Member REQUEST → refund row queued (pending)
                 ↓
Admin opens PayMongo dashboard →
        finds original payment →
        clicks refund → confirms amount →
                 ↓
Admin returns to /admin/refunds →
        pastes PayMongo refund ID → marks completed →
                 ↓
Ticket tier_id auto-swaps via the [tier_change_target] marker.
```

**After step 1 — tier downgrade (1 click):**

```
Member REQUEST → refund row queued (pending)
                 ↓
Admin clicks PROCESS VIA PAYMONGO on the row →
        hard confirm dialog → we call PayMongo /v1/refunds →
        row moves to "processing" → PayMongo webhook fires
                 ↓
Row moves to "completed" → ticket tier_id auto-swaps →
Member gets email + in-app notif "Your refund has been sent."
```

**After step 2 — ticket cancel (1 click):**

```
Admin clicks CANCEL on a ticket row →
        confirm dialog → orchestrator fires:
                 ↓
    - ticket status = cancelled
    - PayMongo refund initiated (via step 1)
    - promoteNextWaitlist(eventId, 1) → next in line notified
    - member gets branded cancellation email
    - capacity count updated
                 ↓
Refund webhook eventually confirms → refund row completed.
```

## NOT IN THIS TRACK (PARKED)

- **Vouchers / store credit** — alternative to real refunds for downgrades. Fan-friendly, big feature. Revisit if PayMongo refunds are unreliable.
- **Refund analytics for finance** — dashboards, cohort views. Step 4 is the MVP; deeper reporting waits until you have volume.
- **Programmable "no refund" windows** — e.g. no refunds < 48h before an event. Simple flag on `events`, but not a real problem until we see abuse.
- **Refund fee reconciliation** — PayMongo takes a cut on original charge; who eats it on refund? Answer once volume matters.

## DEPENDENCIES & MIGRATIONS

- `supabase/migrations/20260905_refunds_event_ticket.sql` — widens the `refunds.entity_type` CHECK to allow `'event_ticket'`. **Not yet applied on remote.** Blocks the downgrade path only; upgrades and same-price swaps work without it.
- PayMongo Refunds API must be enabled on the merchant account before step 1 goes live in production. Email support to confirm.
- Nothing else outstanding.